



Université
de Rennes



Baptiste Cojean

ESIR 2

Compte rendu

The ESIR-as-a-Service Startup

Sommaire :

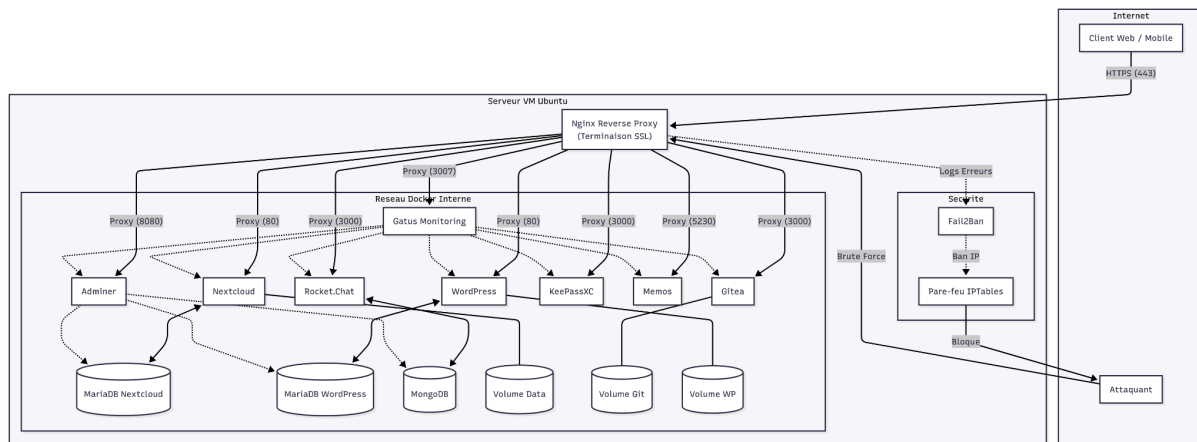
Sommaire :	2
Introduction :	2
Architecture globale :	3
Schéma d'architecture :.....	3
Choix techniques :.....	3
État des services :	4
Justification des choix :	5
Sécurité : HTTPS, Auth & Fail2Ban.....	7
Sécurité.....	7
Chiffrement HTTPS Intégral.....	7
Fail2Ban.....	7
Basic Authentication.....	7
Load testing.....	8
Feedback et conclusion.....	9

Introduction :

Ce projet vise à déployer une infrastructure conteneurisée via Docker. L'objectif est de mettre en place une suite de services (communication, stockage, développement) hébergés sur une machine virtuelle de l'ISTIC, en assurant leur sécurité, leur persistance et leur monitoring.

Architecture globale :

Schéma d'architecture :



Choix techniques :

J'ai choisi d'utiliser Docker pour la conteneurisation des différents services. N'ayant aucune expérience préalable à la conteneurisation, Docker était l'option recommandée et donc le meilleur choix pour mener à bien ce projet.

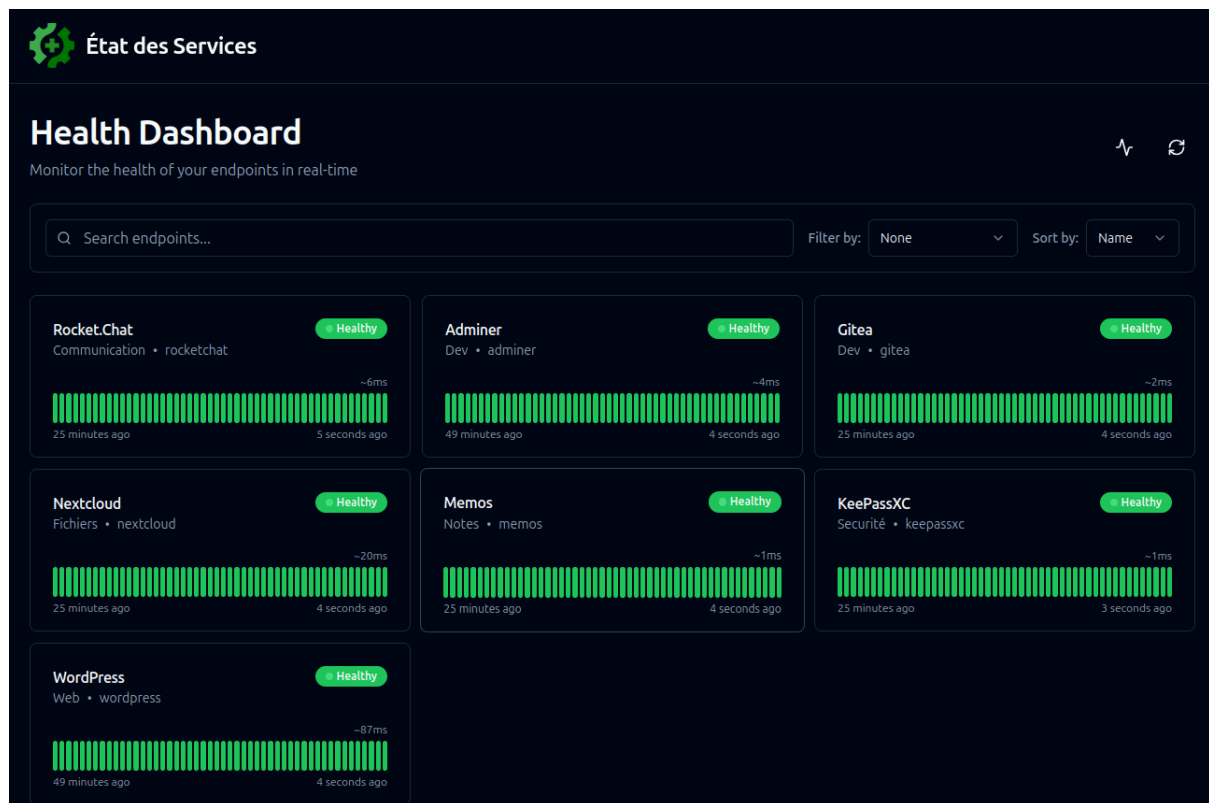
Pour les mêmes raisons, j'ai utilisé Nginx en reverse proxy. J'ai choisi d'installer Nginx directement sur la VM et non en conteneur pour simplifier la gestion des certificats SSL et l'intégration avec Fail2Ban. Nginx gère donc la terminaison SSL, le routage vers les conteneurs et l'injection d'en-têtes de sécurité.

Toutes les données importantes (bases de données, fichiers utilisateurs) sont stockées dans des volumes Docker nommés (ex: gitea_data, mongo_data, ...) pour être conservées à la suppression des conteneurs.

État des services :

Service	Catégorie	Statut	Sécurité (HTTPS)	Remarques
Nginx	Infrastructure	Fonctionnel	Oui	Point d'entrée unique
Nextcloud	Fichiers / Photos	Fonctionnel	Oui	Stockage et partage de fichiers
Rocket.Chat	Messagerie	Fonctionnel	Oui	Configuré avec WebSockets
Gitea	Gestion de code	Fonctionnel	Oui	Alternative légère à GitLab
WordPress	CMS / Web	Fonctionnel	Oui + Basic Auth	Site vitrine protégé
KeePassXC	Mots de passe	Fonctionnel	Oui	Accès Web VNC sécurisé
Memos	Notes	Fonctionnel	Oui	Prise de notes rapide
Adminer	Administration BDD	Fonctionnel	Oui	Gestionnaire MySQL/Mongo (Service Bonus)
Gatus	Monitoring	Fonctionnel	Oui	Surveillance active des endpoints
Pages	Page d'accueil	Fonctionnel	Oui + Basic Auth	Rassemble les accès de chacun de mes services

On peut vérifier l'état de nos systèmes via le service de monitoring Gatus, voici ci-dessous l'état actuel de chacun de mes services :



On vérifie bien le fonctionnement de chacun d'entre eux.

Justification des choix :

Voici les raisons pour lesquels j'ai choisis d'utiliser ces services plutôt que d'autres :

- RocketChat :

J'ai choisi Rocket.Chat pour la messagerie interne car c'est une solution open-source complète et autohébergeable, ce qui garantit un contrôle total sur les données échangées. De plus, il s'intègre facilement avec les autres services et fonctionne avec MongoDB, déjà présent dans notre architecture, ce qui simplifie son déploiement et sa maintenance.

- Adminer :

Pour l'administration des bases de données, j'ai préféré Adminer à PhpMyAdmin. La raison principale est sa polyvalence : PhpMyAdmin ne gère que MySQL/MariaDB alors que Adminer peut gérer à la fois les bases SQL (WordPress, Nextcloud) et la base NoSQL (MongoDB pour Rocket.Chat). De plus, il est très rapide et simple à déployer et possède une interface très simple à utiliser.

- **Gitea :**

J'ai choisi Gitea car c'est un service simple et léger ce qui est un critère important pour le bon fonctionnement de notre architecture vue des ressources limitées de notre Machine Virtuelle.

- **Nextcloud :**

J'ai choisi Nextcloud pour le stockage de fichiers car il est très complet. Il permet également de stocker des photos ce qui m'a évité de rajouter un service supplémentaire pour cette fonction. Même si Nextcloud est un service assez important, il reste performant au vue des services qu'il propose.

- **Memos :**

Pour la prise de notes, Memos est un outil très simple et moderne. Il permet de prendre des notes rapidement et est utile pour des plus petits documents contrairement à Nextcloud.

- **KeePassXC :**

J'ai choisi KeePass pour la gestion de mot de passe car j'avais déjà eu l'occasion d'utiliser ce service lors d'un stage. Je maîtrisais donc l'outil en lui-même et il m'a donc paru intéressant de le déployer

.

- **WordPress :**

WordPress est très facile d'utilisation et permet donc à tous de l'utiliser facilement. Il était aussi un excellent service pour mettre en œuvre des mesures de sécurité (Basic Auth via Nginx, Fail2Ban) car il ne possède pas sa propre méthode de connexion en interne contrairement à la majorité des autres services.

- **Gatus :**

Après des difficultés avec Uptime Kuma, je me suis orienté vers Gatus car il a une approche "configuration as code" ce qui rend sa configuration plus libre et plus simple. Cela permet également de versionner la stratégie de surveillance dans Git et de redéployer le monitoring instantanément sans perte de données.

J'ai choisi d'installer mes services en "hosting as a path" car c'est ce qui m'a semblé le plus simple au départ. C'était effectivement relativement simple de configurer mes services en reverse proxy via un hosting as a path, cependant, de nombreux services ne supportent pas cette façon de faire. Par exemple, les outils de monitoring comme Gatus et Uptime Kuma ne fonctionnent pas du tout via un hosting as a path c'est pourquoi j'ai dû exceptionnellement passer mon service Gatus en hosting as a subdomain.

Sécurité : HTTPS, Auth & Fail2Ban

Sécurité

Chiffrement HTTPS Intégral

L'ensemble du trafic est chiffré via SSL/TLS. Nginx force la redirection de HTTP (port 80) vers HTTPS (port 443) avec des certificats auto-signés générés via OpenSSL.

Fail2Ban

Fail2Ban a été installé sur l'hôte pour protéger les services contre les attaques par force brute.

- Prison [sshd] : Protège l'accès SSH à la VM.
- Prison [nginx-http-auth] : Surveille les logs d'erreurs Nginx et bannit automatiquement les IP après 3 échecs d'authentification sur les pages protégées (Basic Auth).

La capture ci-dessous nous fournit l'état de la prison, ici vide.

```
baptiste@baptiste:~/esir-as-a-service-2025$ sudo fail2ban-client status nginx-http-auth
Status for the jail: nginx-http-auth
|- Filter
| |- Currently failed: 0
| |- Total failed: 0
| `-- Journal matches: _SYSTEMD_UNIT=nginx.service + _COMM=nginx
`- Actions
  |- Currently banned: 0
  |- Total banned: 0
  `-- Banned IP list:
```

Basic Authentication

Une couche de sécurité "Basic Auth" a été ajoutée via Nginx pour restreindre l'accès à la page d'accueil ainsi qu'à Wordpress, protégeant ainsi ces points d'entrée avant même l'accès à l'application. Il n'est pas utile de mettre cette protection à tous les services car ils possèdent leur propre méthode d'authentification.

Load testing

J'ai choisi **K6** pour réaliser mes tests de charge car c'est un outil moderne, scriptable en JavaScript et facilement exécutable avec Docker.

J'ai donc simulé une charge de 10 utilisateurs sur ma page d'accueil pendant 30 secondes, on obtient les résultats suivants :

```
■ TOTAL RESULTS

HTTP
http_req_duration.....: avg=5.36ms min=2.09ms med=3.16ms max=47.01ms p(90)=8.9ms p(95)=13.82ms
{ expected_response:true }...: avg=5.36ms min=2.09ms med=3.16ms max=47.01ms p(90)=8.9ms p(95)=13.82ms
http_req_failed.....: 0.00% 0 out of 300
http_reqs.....: 300 9.929126/s

EXECUTION
iteration_duration.....: avg=1s min=1s med=1s max=1.05s p(90)=1s p(95)=1.01s
iterations.....: 300 9.929126/s
vus.....: 10 min=10 max=10
vus_max.....: 10 min=10 max=10

NETWORK
data_received.....: 4.5 MB 148 kB/s
data_sent.....: 50 kB 1.7 kB/s
```

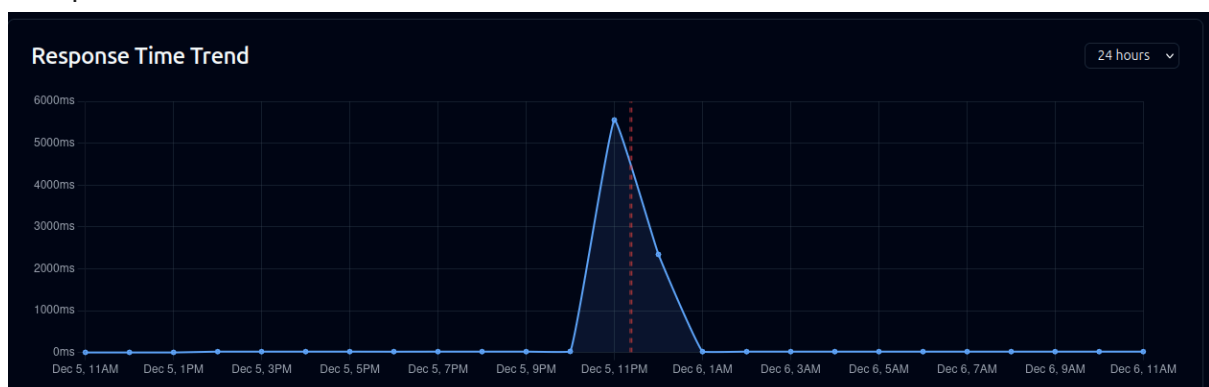
On remarque avec ce faible nombre d'utilisateurs qu'aucune requête n'a échoué, aucune connexion n'a été rejetée.

On remarque une faible latence moyenne de 5.36ms par requête.

Même dans le pire des cas, le serveur reste très réactif (< 50ms).

Le test, réalisé avec 10 utilisateurs virtuels simultanés sur une durée de 30 secondes, montre que l'architecture Nginx + Docker supporte parfaitement la charge sans erreur et avec une latence négligeable.

Ce scénario n'incluant que 10 utilisateurs reste peu représentatif. On teste donc avec un plus grand nombre d'utilisateurs, cette fois-ci 10 000 utilisateurs. On remarque que ce nombre d'utilisateur ralentit fortement l'ensemble de mon architecture jusqu'à rendre inutilisables certains services. Par exemple on constate sur Gatus l'apparition d'un pic de latence :



Aucun load balancing n'ayant été mis en place, cette réaction était totalement prévisible.

Feedback et conclusion

Durant ce TP, j'ai appris le rôle de Nginx. La configuration du Reverse Proxy m'a posé de nombreuses difficultés, notamment pour gérer les WebSockets de Rocket.Chat ou les redirections HTTPS. C'est en devant déboguer les connexions (avec des outils comme curl directement depuis les conteneurs) que j'ai saisi la différence entre le réseau de la machine hôte et le réseau interne de Docker.

Un point important a été la mise en place du monitoring avec Gatus. Cela m'a permis d'utiliser une "Configuration as Code" via les fichiers YAML, que je trouve maintenant plus propre et maintenable que les configurations manuelles avec interface graphique. Cependant, l'intégration de Gatus a aussi été ma plus grosse difficulté : faire fonctionner une Single Page Application (SPA) dans un sous-dossier (/gatus/) posait trop de problèmes de routage. J'ai donc dû choisir de passer ce service en subdomain et non en path comme les autres.

Les 8 services demandés tournent et les données sont persistantes grâce aux volumes Docker. La sécurité est assurée à plusieurs niveaux (HTTPS forcé, Fail2Ban, et Basic Auth). Les tests de charge avec K6 ont validé que le serveur tenait avec un petit nombre d'utilisateurs, même si l'absence de Load Balancing montre ses limites sous forte charge.

Plusieurs améliorations sont encore possibles, notamment séparer chacun de mes services en plusieurs fichiers docker-compose tout comme pour reverse-proxy. Également, l'utilisation de certificats auto-signés pour SSL n'est pas idéale et pourrait être améliorée en utilisant Let's Encrypt pour obtenir des certificats valides reconnus par les navigateurs.